

- Description

commons-math-4.0-beta1-src\commons-math-legacy\src\main\java\org\apache\commons\math4\legacy\analysis\polynomials\PolynomialFunction.java

- Specification

module PolynomialFunction

type

```

c: sequence of real           // input coefficients (c[0] is constant term)
coeffs: sequence of real      // internal stored coefficients (copy of c)
x: real                       // scalar evaluation point
y: real                       // scalar value result
deg: int                      // degree reported by degree()
d_coeffs: sequence of real    // coefficients of derivative polynomial
ex: set of {
    None,
    NullArgumentException,
    NoDataException
}

```

process Construct(c: sequence of real) result: coeffs: sequence of real, deg: int, ex: set of { ... }

Functional Scenario Form

```

    c < 0                                && ex = NullArgumentException ||
|c| = 0                                && ex = NoDataException ||
|c| >= 1                                && (ex = None && coeffs = copy(c) && deg = |c| -
1)
end_process

```

process ValueAt(x: real, coeffs: sequence of real) result: y: real

Functional Scenario Form

```

(|coeffs| = 1)                                && (y = coeffs[0]) or
(|coeffs| > 1 && x = 0)                        && (y = coeffs[0]) or
(exists i: 0 ≤ i < |coeffs| . isNaN(coeffs[i])) && (isNaN(y)) or
(isNaN(x))                                    && (isNaN(y)) or
(isFinite(x) && ∀i . isFinite(coeffs[i]))        && (y computed by Horner; may
under/overflow per IEEE-754) or
(x = +∞ && leading_nonzero(coeffs) > 0)          && (y = +∞) or
(x = +∞ && leading_nonzero(coeffs) < 0)          && (y = -∞) or
(x = -∞ && highest_degree_is_even(coeffs) && leading_nonzero(coeffs) > 0) && (y
= +∞) or
(x = -∞ && highest_degree_is_even(coeffs) && leading_nonzero(coeffs) < 0) && (y
= -∞) or

```

```

    (x = -∞ && highest_degree_is_odd(coeffs) && leading_nonzero(coeffs) > 0) &&
(y = -∞) or
    (x = -∞ && highest_degree_is_odd(coeffs) && leading_nonzero(coeffs) < 0) &&
(y = +∞) or
    (allZero(coeffs))
semantics)
end_process

```

process ValueAtDS(t: DerivativeStructure, coeffs: sequence of real) result: t_val: DerivativeStructure

Functional Scenario Form

```

    (t has NaN component)
    (exists i . isNaN(coeffs[i]))
    (otherwise)
on DS; derivatives propagated)
end_process

```

process Degree(coeffs: sequence of real) result: deg: int

Functional Scenario Form

```

(|coeffs| ≥ 1)
end_process

```

process Derivative(coeffs: sequence of real) result: d_coeffs: sequence of real

Functional Scenario Form

```

(|coeffs| = 1)
(|coeffs| > 1)
coeffs[k+1] for k = 0..|coeffs|-2)
end_process

```

process GetCoefficients(coeffs: sequence of real) result: out: sequence of real

Functional Scenario Form

```

(true)
end_process
end_module

```

- Java Code

```

/*
 * Licensed to the Apache Software Foundation (ASF) under one or more
 * contributor license agreements. See the NOTICE file distributed with

```

```

* this work for additional information regarding copyright ownership.
* The ASF licenses this file to You under the Apache License, Version
2.0
* (the "License"); you may not use this file except in compliance with
* the License. You may obtain a copy of the License at
*
*     http://www.apache.org/licenses/LICENSE-2.0
*
* Unless required by applicable law or agreed to in writing, software
* distributed under the License is distributed on an "AS IS" BASIS,
* WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or impl
ied.
* See the License for the specific language governing permissions and
* limitations under the License.
*/
package org.apache.commons.math4.legacy.analysis.polynomials;
import java.util.Arrays;
import org.apache.commons.math4.legacy.analysis.ParametricUnivariateFun
ction;
import org.apache.commons.math4.legacy.analysis.differentiation.Derivat
iveStructure;
import org.apache.commons.math4.legacy.analysis.differentiation.Univari
ateDifferentiableFunction;
import org.apache.commons.math4.legacy.exception.NoDataException;
import org.apache.commons.math4.legacy.exception.NullArgumentException;
import org.apache.commons.math4.legacy.exception.util.LocalizedFormats;
import org.apache.commons.math4.core.jdkmath.JdkMath;
/**
 * Immutable representation of a real polynomial function with real
 * coefficients.
 * <p>
 * <a href="http://mathworld.wolfram.com/HornersMethod.html">Horner's M
ethod</a>
 * is used to evaluate the function.
 * </p>
 *
 */
public class PolynomialFunction implements UnivariateDifferentiableFunc
tion {
    /**
     * The coefficients of the polynomial, ordered by degree -- i.e.,
     * coefficients[0] is the constant term and coefficients[n] is the coe
fficient
     * of  $x^n$  where n is the degree of the polynomial.

```

```

*/
private final double[] coefficients;
/**
 * Construct a polynomial with the given coefficients. The first element of the
 * coefficients array is the constant term. Higher degree coefficients follow in
 * sequence. The degree of the resulting polynomial is the index of the last
 * non-null element of the array, or 0 if all elements are null.
 * <p>
 * The constructor makes a copy of the input array and assigns the copy to the
 * coefficients property.
 * </p>
 *
 * @param c Polynomial coefficients.
 * @throws NullPointerException if {@code c} is {@code null}.
 * @throws NoDataException      if {@code c} is empty.
 */
public PolynomialFunction(double[] c) throws NullPointerException, NoDataException {
    super();
    NullPointerException.checkNotNull(c);
    int n = c.length;
    if (n == 0) {
        throw new NoDataException(LocalizedFormats.EMPTY_POLYNOMIALS_COEFFICIENTS_ARRAY);
    }
    while (n > 1 && c[n - 1] == 0) {
        --n;
    }
    this.coefficients = new double[n];
    System.arraycopy(c, 0, this.coefficients, 0, n);
}
/**
 * Compute the value of the function for the given argument.
 * <p>
 * The value returned is
 * </p>
 * <p>
 * {@code coefficients[n] * x^n + ... + coefficients[1] * x + coefficients[0]}
 * </p>

```

```

*
* @param x Argument for which the function value should be computed.
* @return the value of the polynomial at the given point.
*
* @see org.apache.commons.math4.legacy.analysis.UnivariateFunction#value(double)
*/
@Override
public double value(double x) {
    return evaluate(coefficients, x);
}
/**
 * Returns the degree of the polynomial.
 *
 * @return the degree of the polynomial.
 */
public int degree() {
    return coefficients.length - 1;
}
/**
 * Returns a copy of the coefficients array.
 * <p>
 * Changes made to the returned copy will not affect the coefficients
of the
 * polynomial.
 * </p>
 *
 * @return a fresh copy of the coefficients array.
 */
public double[] getCoefficients() {
    return coefficients.clone();
}
/**
 * Uses Horner's Method to evaluate the polynomial with the given coefficients
at the argument.
 *
 * @param coefficients Coefficients of the polynomial to evaluate.
 * @param argument Input value.
 * @return the value of the polynomial.
 * @throws NoDataException if {@code coefficients} is empty.
 * @throws NullPointerException if {@code coefficients} is {@code null}.
 */

```

```

protected static double evaluate(double[] coefficients, double argument)
    throws NullPointerException, NoDataException {
    NullPointerException.checkNotNull(coefficients);
    int n = coefficients.length;
    if (n == 0) {
        throw new NoDataException(LocalizedFormats.EMPTY_POLYNOMIALS_COEFFICIENTS_ARRAY);
    }
    double result = coefficients[n - 1];
    for (int j = n - 2; j >= 0; j--) {
        result = argument * result + coefficients[j];
    }
    return result;
}

/**
 * {@inheritDoc}
 *
 * @since 3.1
 * @throws NoDataException if {@code coefficients} is empty.
 * @throws NullPointerException if {@code coefficients} is {@code null}.
 */
@Override
public DerivativeStructure value(final DerivativeStructure t) throws NullPointerException, NoDataException {
    NullPointerException.checkNotNull(coefficients);
    int n = coefficients.length;
    if (n == 0) {
        throw new NoDataException(LocalizedFormats.EMPTY_POLYNOMIALS_COEFFICIENTS_ARRAY);
    }
    DerivativeStructure result = new DerivativeStructure(t.getFreeParameters(), t.getOrder(), coefficients[n - 1]);
    for (int j = n - 2; j >= 0; j--) {
        result = result.multiply(t).add(coefficients[j]);
    }
    return result;
}

/**
 * Add a polynomial to the instance.
 *
 * @param p Polynomial to add.

```

```

    * @return a new polynomial which is the sum of the instance and {@code p}.
    */
    public PolynomialFunction add(final PolynomialFunction p) {
        // identify the lowest degree polynomial
        final int lowLength = JdkMath.min(coefficients.length, p.coefficients.length);
        final int highLength = JdkMath.max(coefficients.length, p.coefficients.length);
        // build the coefficients array
        double[] newCoefficients = new double[highLength];
        for (int i = 0; i < lowLength; ++i) {
            newCoefficients[i] = coefficients[i] + p.coefficients[i];
        }
        System.arraycopy((coefficients.length < p.coefficients.length) ? p.coefficients : coefficients, lowLength,
            newCoefficients, lowLength, highLength - lowLength);
        return new PolynomialFunction(newCoefficients);
    }
    /**
     * Subtract a polynomial from the instance.
     *
     * @param p Polynomial to subtract.
     * @return a new polynomial which is the instance minus {@code p}.
     */
    public PolynomialFunction subtract(final PolynomialFunction p) {
        // identify the lowest degree polynomial
        int lowLength = JdkMath.min(coefficients.length, p.coefficients.length);
        int highLength = JdkMath.max(coefficients.length, p.coefficients.length);
        // build the coefficients array
        double[] newCoefficients = new double[highLength];
        for (int i = 0; i < lowLength; ++i) {
            newCoefficients[i] = coefficients[i] - p.coefficients[i];
        }
        if (coefficients.length < p.coefficients.length) {
            for (int i = lowLength; i < highLength; ++i) {
                newCoefficients[i] = -p.coefficients[i];
            }
        } else {
            System.arraycopy(coefficients, lowLength, newCoefficients, lowLength,
                highLength - lowLength);
        }
    }

```

```

    return new PolynomialFunction(newCoefficients);
}
/**
 * Negate the instance.
 *
 * @return a new polynomial with all coefficients negated
 */
public PolynomialFunction negate() {
    double[] newCoefficients = new double[coefficients.length];
    for (int i = 0; i < coefficients.length; ++i) {
        newCoefficients[i] = -coefficients[i];
    }
    return new PolynomialFunction(newCoefficients);
}
/**
 * Multiply the instance by a polynomial.
 *
 * @param p Polynomial to multiply by.
 * @return a new polynomial equal to this times {@code p}
 */
public PolynomialFunction multiply(final PolynomialFunction p) {
    double[] newCoefficients = new double[coefficients.length + p.coefficients.length - 1];
    for (int i = 0; i < newCoefficients.length; ++i) {
        newCoefficients[i] = 0.0;
        for (int j = JdkMath.max(0, i + 1 - p.coefficients.length); j < JdkMath.min(coefficients.length, i + 1); ++j) {
            newCoefficients[i] += coefficients[j] * p.coefficients[i - j];
        }
    }
    return new PolynomialFunction(newCoefficients);
}
/**
 * Returns the coefficients of the derivative of the polynomial with the given
 * coefficients.
 *
 * @param coefficients Coefficients of the polynomial to differentiate
 *
 * @return the coefficients of the derivative or {@code null} if coefficients
 *         has length 1.
 * @throws NoDataException if {@code coefficients} is empty.

```

```

    * @throws NullPointerException if {@code coefficients} is {@code nul
    l}.
    */
    protected static double[] differentiate(double[] coefficients) throws
    NullPointerException, NoDataException {
        NullPointerException.checkNotNull(coefficients);
        int n = coefficients.length;
        if (n == 0) {
            throw new NoDataException(LocalizedFormats.EMPTY_POLYNOMIALS_COEFFIC
            IENTS_ARRAY);
        }
        if (n == 1) {
            return new double[] { 0 };
        }
        double[] result = new double[n - 1];
        for (int i = n - 1; i > 0; i--) {
            result[i - 1] = i * coefficients[i];
        }
        return result;
    }
    /**
     * Returns the derivative as a {@link PolynomialFunction}.
     *
     * @return the derivative polynomial.
     */
    public PolynomialFunction polynomialDerivative() {
        return new PolynomialFunction(differentiate(coefficients));
    }
    /**
     * Returns a string representation of the polynomial.
     *
     * <p>
     * The representation is user oriented. Terms are displayed lowest deg
     rees
     * first. The multiplications signs, coefficients equals to one and nu
     ll terms
     * are not displayed (except if the polynomial is 0, in which case the
     0
     * constant term is displayed). Addition of terms with negative coeffi
     cients are
     * replaced by subtraction of terms with positive coefficients except
     for the
     * first displayed term (i.e. we display <code>-
     3</code> for a constant negative

```

```

    * polynomial, but 1 - 3 x + x^2 if the negative coefficient is not
    * the first one displayed).
    * </p>
    *
    * @return a string representation of the polynomial.
    */
    @Override
    public String toString() {
        StringBuilder s = new StringBuilder();
        if (coefficients[0] == 0.0) {
            if (coefficients.length == 1) {
                return "0";
            }
        } else {
            s.append(toString(coefficients[0]));
        }
        for (int i = 1; i < coefficients.length; ++i) {
            if (coefficients[i] != 0) {
                if (s.length() > 0) {
                    if (coefficients[i] < 0) {
                        s.append(" - ");
                    } else {
                        s.append(" + ");
                    }
                } else {
                    if (coefficients[i] < 0) {
                        s.append("-");
                    }
                }
                double absAi = JdkMath.abs(coefficients[i]);
                if ((absAi - 1) != 0) {
                    s.append(toString(absAi));
                    s.append(' ');
                }
                s.append("x");
                if (i > 1) {
                    s.append('^');
                    s.append(Integer.toString(i));
                }
            }
        }
        return s.toString();
    }
}

```

```

/**
 * Creates a string representing a coefficient, removing ".0" endings.
 *
 * @param coeff Coefficient.
 * @return a string representation of {@code coeff}.
 */
private static String toString(double coeff) {
    final String c = Double.toString(coeff);
    if (c.endsWith(".0")) {
        return c.substring(0, c.length() - 2);
    } else {
        return c;
    }
}

/** {@inheritDoc} */
@Override
public int hashCode() {
    final int prime = 31;
    int result = 1;
    result = prime * result + Arrays.hashCode(coefficients);
    return result;
}

/** {@inheritDoc} */
@Override
public boolean equals(Object obj) {
    if (this == obj) {
        return true;
    }
    if (!(obj instanceof PolynomialFunction)) {
        return false;
    }
    PolynomialFunction other = (PolynomialFunction) obj;
    return Arrays.equals(coefficients, other.coefficients);
}

/**
 * Dedicated parametric polynomial class.
 *
 * @since 3.0
 */
public static class Parametric implements ParametricUnivariateFunction
{
    /** {@inheritDoc} */
    @Override
    public double[] gradient(double x, double... parameters) {

```

```

        final double[] gradient = new double[parameters.length];
        double xn = 1.0;
        for (int i = 0; i < parameters.length; ++i) {
            gradient[i] = xn;
            xn *= x;
        }
        return gradient;
    }
    /** {@inheritDoc} */
    @Override
    public double value(final double x, final double... parameters) throws NoDataException {
        return PolynomialFunction.evaluate(parameters, x);
    }
}
//VT
class PFVT {
    private static void title(String name) {
        System.out.println("==== " + name + " =====");
    }
    private static void expectThrow(String name, Runnable r, Class<?> expected) {
        System.out.println("-- " + name + " --");
        try {
            r.run();
            System.out.printf("[FAIL] %s should throw %s\n", name, expected.getSimpleName());
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
            System.out.printf("[%s] %s threw expected\n", t.getClass().equals(expected) ? "PASS" : "FAIL", name);
        }
    }
    private static String fmt(double v) {
        if (Double.isNaN(v))
            return "NaN";
        if (v == Double.POSITIVE_INFINITY)
            return "+Inf";
        if (v == Double.NEGATIVE_INFINITY)
            return "-Inf";
        if (v == 0.0) {
            long bits = Double.doubleToRawLongBits(v);

```

```

        return (bits >>> 63) == 1 ? "-0.0" : "0.0";
    }
    return Double.toString(v);
}
private static void checkEquals(String name, double expected, double actual) {
    boolean ok;
    if (Double.isNaN(expected) || Double.isNaN(actual))
        ok = (Double.isNaN(expected) && Double.isNaN(actual));
    else
        ok = (expected == actual) || (Math.abs(expected - actual) <= 1e-12
            * Math.max(1.0, Math.max(Math.abs(expected), Math.abs(actual))));
    System.out.printf("[%s] %s expected=%s actual=%s\n", ok ? "PASS" : "FAIL", name, fmt(expected), fmt(actual));
}
private static void checkArrayEquals(String name, double[] exp, double[] act) {
    boolean ok = exp.length == act.length;
    if (ok) {
        for (int i = 0; i < exp.length; i++) {
            double a = exp[i], b = act[i];
            if (Double.isNaN(a) || Double.isNaN(b)) {
                ok &= (Double.isNaN(a) && Double.isNaN(b));
            } else if (a == 0d && b == 0d) {
                /* accept signed zeros */ } else if (a == b) {
                /* ok */ } else if (Math.abs(a - b) <= 1e-12
                * Math.max(1.0, Math.max(Math.abs(a), Math.abs(b)))) {
                /* ok */ } else {
                ok = false;
                break;
            }
        }
    }
    System.out.printf("[%s] %s expected=%s actual=%s\n", ok ? "PASS" : "FAIL", name, Arrays.toString(exp), Arrays.toString(act));
}
/* ----- VT cases ----- */
// VT01: ctor null
private static void VT01() {
    title("VT01 ctor null → NullArgumentException");
    expectThrow("ctor(null)", () -> new PolynomialFunction((double[]) null), NullArgumentException.class);
}

```

```

// VT02: ctor empty
private static void VT02() {
    title("VT02 ctor empty → NoDataException");
    expectThrow("ctor(empty)", () -> new PolynomialFunction(new double[]
{}), NoDataException.class);
}

// VT03: constant poly + degree + derivative {0}
private static void VT03() {
    title("VT03 constant {3.0}");
    PolynomialFunction p = new PolynomialFunction(new double[] { 3.0 });
    checkEquals("degree()", 0, p.degree());
    checkEquals("p(0)", 3.0, p.value(0.0));
    checkArrayEquals("derivative({3.0})", new double[] { 0.0 }, p.polynom
ialDerivative().getCoefficients());
}

// VT04: linear, x=NaN → NaN
private static void VT04() {
    title("VT04 linear {1,2} x=NaN");
    PolynomialFunction p = new PolynomialFunction(new double[] { 1.0, 2.0
});
    checkEquals("NaN propagates", Double.NaN, p.value(Double.NaN));
}

// VT05: NaN coefficient → NaN
private static void VT05() {
    title("VT05 NaN coefficient {NaN, 2}");
    PolynomialFunction p = new PolynomialFunction(new double[] { Double.N
aN, 2.0 });
    checkEquals("coeff NaN→NaN", Double.NaN, p.value(1.23));
}

// VT06: even degree leading positive, x=±Inf
private static void VT06() {
    title("VT06 even deg leading + : {1,0,1}");
    PolynomialFunction p = new PolynomialFunction(new double[] { 1.0, 0.0
, 1.0 }); // x^2+1
    checkEquals("x=+Inf", Double.POSITIVE_INFINITY, p.value(Double.POSITI
VE_INFINITY));
    checkEquals("x=-
Inf", Double.POSITIVE_INFINITY, p.value(Double.NEGATIVE_INFINITY));
}

// VT07: even degree leading negative, x=±Inf
private static void VT07() {
    title("VT07 even deg leading - : {1,0,-1}");
    PolynomialFunction p = new PolynomialFunction(new double[] { 1.0, 0.0
, -1.0 }); // -x^2+1

```

```

    checkEquals("x=+Inf", Double.NEGATIVE_INFINITY, p.value(Double.POSITIVE_INFINITY));
    checkEquals("x=-
Inf", Double.NEGATIVE_INFINITY, p.value(Double.NEGATIVE_INFINITY));
}
// VT08: odd degree  $\pm$  Leading coeff at  $\pm Inf$ 
private static void VT08() {
    title("VT08 odd degree {0,1} and {0,-1}");
    PolynomialFunction p1 = new PolynomialFunction(new double[] { 0.0, 1.
0 }); // +x
    PolynomialFunction p2 = new PolynomialFunction(new double[] { 0.0, -
1.0 }); // -x
    checkEquals("p1(+Inf)", Double.POSITIVE_INFINITY, p1.value(Double.POSITIVE_INFINITY));
    checkEquals("p1(-
Inf)", Double.NEGATIVE_INFINITY, p1.value(Double.NEGATIVE_INFINITY));
    checkEquals("p2(+Inf)", Double.NEGATIVE_INFINITY, p2.value(Double.POSITIVE_INFINITY));
    checkEquals("p2(-
Inf)", Double.POSITIVE_INFINITY, p2.value(Double.NEGATIVE_INFINITY));
}
// VT09: all zeros {0,0,0} (degree=length-
1), derivative {0,0}, value==0 for any
// x
private static void VT09() {
    title("VT09 all-zero {0,0,0}");
    PolynomialFunction p = new PolynomialFunction(new double[] { 0.0, 0.0
, 0.0 });
    checkEquals("degree()", 2, p.degree());
    checkEquals("p(0)", 0.0, p.value(0.0));
    checkEquals("p(-123)", 0.0, p.value(-123.0));
    checkArrayEquals("derivative", new double[] { 0.0, 0.0 }, p.polynomialDerivative().getCoefficients());
}
// VT10: overflow to +Inf
private static void VT10() {
    title("VT10 overflow: {0,0,1e308} at x=1e154");
    PolynomialFunction p = new PolynomialFunction(new double[] { 0.0, 0.0
, 1e308 });
    checkEquals("overflow", Double.POSITIVE_INFINITY, p.value(1e154));
}
// VT11: underflow to 0.0
private static void VT11() {
    title("VT11 underflow: {0,0,1e-308} at x=1e-308");

```

```

    PolynomialFunction p = new PolynomialFunction(new double[] { 0.0, 0.0
, 1e-308 });
    checkEquals("underflow→0", 0.0, p.value(1e-308));
}
// VT12: general derivative check
private static void VT12() {
    title("VT12 derivative general {2,-3,4} -> {-3,8}");
    PolynomialFunction p = new PolynomialFunction(new double[] { 2.0, -
3.0, 4.0 });
    PolynomialFunction dp = p.polynomialDerivative();
    checkArrayEquals("d coeffs", new double[] { -
3.0, 8.0 }, dp.getCoefficients());
    checkEquals("d(0.5)=1", 1.0, dp.value(0.5));
}
// VT13: getCoefficients returns copy; ctor copies source
private static void VT13() {
    title("VT13 copy semantics");
    double[] src = new double[] { 5.0, -1.0, 2.0 };
    PolynomialFunction p = new PolynomialFunction(src);
    double[] out = p.getCoefficients();
    out[0] += 10.0;
    src[0] -= 7.0;
    checkEquals("p.value(0) still 5.0", 5.0, p.value(0.0));
}
// VT14: signed zero behavior at x=0 with c0 = -0.0
private static void VT14() {
    title("VT14 signed zero with c0=-0.0");
    double minusZero = -0.0;
    PolynomialFunction p = new PolynomialFunction(new double[] { minusZer
o, 0.0, 0.0 });
    double y = p.value(0.0);
    long bits = Double.doubleToRawLongBits(y);
    System.out.printf("p(0)=%s raw=0x%016X%n", fmt(y), bits);
    // Accept either sign per implementation, but print to flip branch if
    sign is
    // preserved
    checkEquals("p(0) equals 0.0 numerically", 0.0, y);
}
// VT15: Leading zeros NOT trimmed; degree equals length-
1; p(2) uses Horner
// across zeros
private static void VT15() {
    title("VT15 leading zeros not trimmed {1,0,0,0}");

```

```

    PolynomialFunction p = new PolynomialFunction(new double[] { 1.0, 0.0
, 0.0, 0.0 });
    checkEquals("degree()", 3, p.degree());
    checkEquals("p(2)==1", 1.0, p.value(2.0));
}
// VT16: deep Horner loop with alternating signs to exercise accumulat
ion
// branches
private static void VT16() {
    title("VT16 deep Horner alternating signs");
    double[] c = new double[16];
    for (int i = 0; i < c.length; i++)
        c[i] = (i % 2 == 0) ? 1.0 : -1.0;
    PolynomialFunction p = new PolynomialFunction(c);
    double y1 = p.value(0.99);
    double y2 = p.value(-0.99);
    System.out.printf("p(0.99)=%s, p(-0.99)=%s\n", fmt(y1), fmt(y2));
    // No fixed expected value asserted; purpose is to traverse many Horn
er steps
    // and signs.
    // Still check finite:
    boolean finite = Double.isFinite(y1) && Double.isFinite(y2);
    System.out.printf("[%s] finite outputs\n", finite ? "PASS" : "FAIL");
}
// VT17: DS evaluation (order 1) matches analytic derivative at x=0.5
for
// {2,-3,4}
private static void VT17() {
    title("VT17 DerivativeStructure order1 vs analytic");
    PolynomialFunction p = new PolynomialFunction(new double[] { 2.0, -
3.0, 4.0 }); //  $2 - 3x + 4x^2$ ;  $d = -3 + 8x$ 
    DerivativeStructure x = new DerivativeStructure(1, 1, 0, 0.5); // 1 v
ar, order 1, index 0, value 0.5
    DerivativeStructure y = p.value(x);
    double val = y.getValue();
    double d1 = y.getPartialDerivative(1); //  $dy/dx$ 
    checkEquals("value @0.5", 2.0 - 3.0 * 0.5 + 4.0 * 0.25, val); //  $2 -
1.5 + 1 = 1.5$ 
    checkEquals("d1 @0.5", -3.0 + 8.0 * 0.5, d1); // 1.0
}
// VT18: DS evaluation (order 2) second derivative equals 8 for {2,-
3,4}
private static void VT18() {
    title("VT18 DerivativeStructure order2 second derivative");

```

```

    PolynomialFunction p = new PolynomialFunction(new double[] { 2.0, -
3.0, 4.0 }); // d2 = 8
    DerivativeStructure x = new DerivativeStructure(1, 2, 0, -
1.25); // any x
    DerivativeStructure y = p.value(x);
    double d2 = y.getPartialDerivative(2); // d2y/dx2
    checkEquals("d2 == 8", 8.0, d2);
}
// VT19: DS NaN propagation via x (any coeffs)
private static void VT19() {
    title("VT19 DS NaN via x");
    DerivativeStructure xNaN = new DerivativeStructure(1, 1, 0, Double.Na
N);
    PolynomialFunction p = new PolynomialFunction(new double[] { 1.0, 2.0
});
    DerivativeStructure y = p.value(xNaN);
    double val = y.getValue();
    System.out.println("y.value = " + fmt(val));
    checkEquals("NaN propagated", Double.NaN, val);
}
// VT20: DS NaN via coefficient
private static void VT20() {
    title("VT20 DS NaN via coeff");
    PolynomialFunction p = new PolynomialFunction(new double[] { Double.N
aN, 1.0, 2.0 });
    DerivativeStructure x = new DerivativeStructure(1, 2, 0, 0.3);
    DerivativeStructure y = p.value(x);
    checkEquals("NaN propagated (coeff)", Double.NaN, y.getValue());
}
/* ----- main ----- */
public void Vtest() {
    System.out.println("==== PolynomialFunction • V-
method VT suite =====");
    VT01();
    VT02();
    VT03();
    VT04();
    VT05();
    VT06();
    VT07();
    VT08();
    VT09();
    VT10();
    VT11();
}

```

```

VT12();
VT13();
VT14();
VT15();
VT16();
VT17();
VT18();
VT19();
VT20();
System.out.println("===== Done =====");
}
}
//FT
class PFFT {
    private static String fmt(double v) {
        if (Double.isNaN(v))
            return "NaN";
        if (v == Double.POSITIVE_INFINITY)
            return "+Inf";
        if (v == Double.NEGATIVE_INFINITY)
            return "-Inf";
        if (v == 0.0) {
            long bits = Double.doubleToRawLongBits(v);
            return (bits >>> 63) == 1 ? "-0.0" : "0.0";
        }
        return Double.toString(v);
    }
    private static void runFT(String name, double[] coeffs, double x) {
        System.out.println("---- " + name + " ----");
        System.out.println("coeffs = " + Arrays.toString(coeffs));
        System.out.println("x      = " + fmt(x));
        try {
            PolynomialFunction p = new PolynomialFunction(coeffs);
            double y = p.value(x);
            System.out.printf("degree = %d\n", p.degree());
            System.out.printf("p(x)   = %s\n", fmt(y));
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
        }
    }
    private static void FT01() {
        runFT("FT01", new double[] { -1808425884.0, 1677918208.0, -
3.14159, 2.718281828, -0.57721 }, 1.23456789);
    }
}

```

```

}
private static void FT02() {
    runFT("FT02", new double[] { 1e308, -1e154, 3.0, -2.0, 0.5 }, 1e154);
}
private static void FT03() {
    runFT("FT03", new double[] { 0.0, 1.0, -2.0, 3.0, -
4.0, 5.0 }, Double.MIN_VALUE);
}
private static void FT04() {
    runFT("FT04", new double[] { 1.0, -1.0, 1.0, -1.0, 1.0, -1.0, 1.0, -
1.0 }, 0.9999999999999999);
}
private static void FT05() {
    runFT("FT05", new double[] { 7.0, -6.5, 5.5, -4.5, 3.5, -2.5, 1.5, -
0.5, 0.25, -0.125, 0.0625, -0.03125 },
-0.987654321);
}
private static void FT06() {
    runFT("FT06", new double[] { 1.0, 0.0, 0.0, -
2.0, 0.0, 0.0, 3.0, 0.0, -4.0, 0.0, 5.0 }, 2.0);
}
private static void FT07() {
    runFT("FT07", new double[] { -1e308, 5e307, -2e307, 1e306, -
1e305, 42.0 }, -1e154);
}
private static void FT08() {
    runFT("FT08", new double[] { -0.0, 0.0, 0.0, 0.0 }, 0.0);
}
private static void FT09() {
    runFT("FT09", new double[] { 1234.567, -8901.234, 0.000123, -
0.4567, 89.01, -2.0, 0.3333333333 }, -3.21);
}
private static void FT10() {
    runFT("FT10", new double[] { 1.0, 2.0, -3.0, 4.0, -
5.0 }, Double.MIN_NORMAL);
}
private static void FT11() {
    runFT("FT11", new double[] { 0.5, -1.0, 2.0, -4.0, 8.0, -
16.0, 32.0 }, Double.POSITIVE_INFINITY);
}
private static void FT12() {
    runFT("FT12", new double[] { -3.0, 0.0, 0.0, 7.0, -
11.0, 13.0 }, Double.NEGATIVE_INFINITY);
}

```

```
private static void FT13() {
    runFT("FT13", new double[] { 1e-308, -1e-200, 1e-100, -1e-50, 1e-
10, -1e10, 1e50, -1e100, 1e200, -1e308 },
        0.75);
}

private static void FT14() {
    runFT("FT14", new double[] { Math.PI, -
Math.E, 1.0 / Math.sqrt(2.0), -0.0, 42.0 }, Math.PI);
}

private static void FT15() {
    runFT("FT15", new double[] { -123456789.25 }, -1e20);
}

private static void FT16() {
    runFT("FT16", new double[] { 2.0, -3.0, 5.0, -7.0, 11.0, -
13.0, 17.0, -19.0, 23.0, -29.0, 31.0, -37.0, 41.0 },
        -0.9999999);
}

private static void FT17() {
    runFT("FT17", new double[] { 987654321.0, -
2147483648.0, 135791113.0, -24681012.0, 1123581321.0 }, 1.5);
}

private static void FT18() {
    runFT("FT18", new double[] { Double.NaN, 1.0, Double.POSITIVE_INFINITY,
-2.0, 3.0 }, 2.0);
}

private static void FT19() {
    runFT("FT19", new double[] { 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1 }, 0.1);
}

private static void FT20() {
    runFT("FT20", new double[] { 1e-200, -1e-210, 1e-220, -1e-230, 1e-
240, -1e-250, 1e-260, -1e-270 }, -1e-154);
}

public void Ftest() {
    System.out.println("=====PolynomialFunction • Fuzzing (FT01-FT20) =====");
    FT01();
    FT02();
    FT03();
    FT04();
    FT05();
    FT06();
    FT07();
}
```

```

    FT08();
    FT09();
    FT10();
    FT11();
    FT12();
    FT13();
    FT14();
    FT15();
    FT16();
    FT17();
    FT18();
    FT19();
    FT20();
    System.out.println("===== Done =====");
}
}
//Z3
class PFZ3 {
    // ----- helpers -----
    private static void headline(String name) {
        System.out.println("\n==== " + name + " ===");
    }
    private static void expectThrow(String name, Runnable r, Class<?> expected) {
        System.out.println("---- " + name + " ----");
        try {
            r.run();
            System.out.printf("[FAIL] %s should throw %s\n", name, expected.getSimpleName());
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
            System.out.printf("[%s] %s threw expected\n", t.getClass().equals(expected) ? "PASS" : "FAIL", name);
        }
    }
    private static void checkEquals(String name, double expected, double actual) {
        boolean ok;
        if (Double.isNaN(expected) || Double.isNaN(actual)) {
            ok = (Double.isNaN(expected) && Double.isNaN(actual));
        } else if (Double.isInfinite(expected) || Double.isInfinite(actual)) {

```

```

    ok = (Double.doubleToRawLongBits(expected) == Double.doubleToRawLong
Bits(actual));
} else {
    double tol = 1e-
12 * Math.max(1.0, Math.max(Math.abs(expected), Math.abs(actual)));
    ok = (expected == actual) || (Math.abs(expected - actual) <= tol);
}
System.out.printf("[%s] %s expected=%s actual=%s\n", ok ? "PASS" : "
FAIL", name, fmt(expected), fmt(actual));
}
private static void checkInt(String name, int expected, int actual) {
    boolean ok = expected == actual;
    System.out.printf("[%s] %s expected=%d actual=%d\n", ok ? "PASS" : "
FAIL", name, expected, actual);
}
private static void checkArrayEquals(String name, double[] exp, double
[] act) {
    boolean ok = exp.length == act.length;
    if (ok) {
        for (int i = 0; i < exp.length; i++) {
            double a = exp[i], b = act[i];
            if (Double.isNaN(a) || Double.isNaN(b)) {
                ok &= (Double.isNaN(a) && Double.isNaN(b));
            } else if (a == 0d && b == 0d) {
                /* accept signed zeros */ } else if (a == b) {
                /* ok */ } else if (Math.abs(a - b) <= 1e-
12 * Math.max(1.0, Math.max(Math.abs(a), Math.abs(b)))) {
                /* ok */ } else {
                ok = false;
                break;
            }
        }
    }
}
System.out.printf("[%s] %s expected=%s actual=%s\n", ok ? "PASS" : "
FAIL", name, Arrays.toString(exp),
Arrays.toString(act));
}
private static String fmt(double v) {
    if (Double.isNaN(v))
        return "NaN";
    if (v == Double.POSITIVE_INFINITY)
        return "+Inf";
    if (v == Double.NEGATIVE_INFINITY)
        return "-Inf";
}

```

```

    if (v == 0.0) {
        long bits = Double.doubleToRawLongBits(v);
        return (bits >>> 63) == 1 ? "-0.0" : "0.0";
    }
    return Double.toString(v);
}

// ----- Z3 tests -----
private static void Z301() {
    headline("Z301 Construct: c=null → NullArgumentException");
    expectThrow("ctor(null)", () -> new PolynomialFunction((double[]) null), NullArgumentException.class);
}

private static void Z302() {
    headline("Z302 Construct: |c|=0 → NoDataException");
    expectThrow("ctor(empty)", () -> new PolynomialFunction(new double[] {}), NoDataException.class);
}

private static void Z303() {
    headline("Z303 Construct: |c|≥1 → deg=|c|-1");
    double[] c = { 2.0, -1.0, 3.0 };
    PolynomialFunction p = new PolynomialFunction(c);
    checkInt("degree", 2, p.degree());
    // 确认 ctor 拷贝: 修改 c 不影响 p
    c[0] = 999.0;
    checkEquals("value(0) uses original c0", 2.0, p.value(0.0));
}

private static void Z304() {
    headline("Z304 ValueAt: |coeffs|=1 → y=c0");
    PolynomialFunction p = new PolynomialFunction(new double[] { 2.0 });
    checkInt("degree", 0, p.degree());
    checkEquals("p(123)", 2.0, p.value(123.0));
}

private static void Z305() {
    headline("Z305 ValueAt: |coeffs|>1 & x=0 → y=c0");
    PolynomialFunction p = new PolynomialFunction(new double[] { 2.0, -1.0 });
    checkEquals("p(0)", 2.0, p.value(0.0));
}

private static void Z306() {
    headline("Z306 ValueAt: coeffs has NaN → y=NaN");
    PolynomialFunction p = new PolynomialFunction(new double[] { Double.NaN, 2.0 });
    checkEquals("p(1.23)", Double.NaN, p.value(1.23));
}

```

```

private static void Z307() {
    headline("Z307 ValueAt: x is NaN → y=NaN");
    PolynomialFunction p = new PolynomialFunction(new double[] { 1.0, 2.0
});
    checkEquals("p(NaN)", Double.NaN, p.value(Double.NaN));
}
private static void Z308() {
    headline("Z308 ValueAt: finite x & finite coeffs → Horner");
    PolynomialFunction p = new PolynomialFunction(new double[] { 1.0, -
3.0, 2.0 }); // 1 - 3x + 2x^2
    double x = 0.75;
    double expected = 1.0 - 3.0 * x + 2.0 * x * x;
    checkEquals("p(0.75)", expected, p.value(x));
}
private static void Z309() {
    headline("Z309 ValueAt: x=+Inf & leading>0 → +Inf");
    PolynomialFunction p = new PolynomialFunction(new double[] { 1.0, 0.0
, 1.0 }); // x^2+1
    checkEquals("p(+Inf)", Double.POSITIVE_INFINITY, p.value(Double.POSIT
IVE_INFINITY));
}
private static void Z310() {
    headline("Z310 ValueAt: x=+Inf & leading<0 → -Inf");
    PolynomialFunction p = new PolynomialFunction(new double[] { 1.0, 0.0
, -1.0 }); // -x^2+1
    checkEquals("p(+Inf)", Double.NEGATIVE_INFINITY, p.value(Double.POSIT
IVE_INFINITY));
}
private static void Z311() {
    headline("Z311 ValueAt: x=-Inf & even deg & leading>0 → +Inf");
    PolynomialFunction p = new PolynomialFunction(new double[] { 1.0, 0.0
, 1.0 }); // x^2+1
    checkEquals("p(-
Inf)", Double.POSITIVE_INFINITY, p.value(Double.NEGATIVE_INFINITY));
}
private static void Z312() {
    headline("Z312 ValueAt: x=-Inf & even deg & leading<0 → -Inf");
    PolynomialFunction p = new PolynomialFunction(new double[] { 1.0, 0.0
, -1.0 }); // -x^2+1
    checkEquals("p(-
Inf)", Double.NEGATIVE_INFINITY, p.value(Double.NEGATIVE_INFINITY));
}
private static void Z313() {
    headline("Z313 ValueAt: x=-Inf & odd deg & leading>0 → -Inf");

```

```

    PolynomialFunction p = new PolynomialFunction(new double[] { 0.0, 1.0
}); // x
    checkEquals("p(-
Inf)", Double.NEGATIVE_INFINITY, p.value(Double.NEGATIVE_INFINITY));
}
private static void Z314() {
    headline("Z314 ValueAt: x=-Inf & odd deg & leading<0 → +Inf");
    PolynomialFunction p = new PolynomialFunction(new double[] { 0.0, -
1.0 }); // -x
    checkEquals("p(-
Inf)", Double.POSITIVE_INFINITY, p.value(Double.NEGATIVE_INFINITY));
}
private static void Z315() {
    headline("Z315 ValueAt: allZero(coeffs) → y=0.0");
    PolynomialFunction p = new PolynomialFunction(new double[] { 0.0, 0.0
, 0.0 });
    checkEquals("p(123)", 0.0, p.value(123.0));
    checkEquals("p(-1e308)", 0.0, p.value(-1e308));
}
private static void Z316() {
    headline("Z316 ValueAtDS: t has NaN → t_val is NaN");
    PolynomialFunction p = new PolynomialFunction(new double[] { 1.0, 2.0
});
    DerivativeStructure xNaN = new DerivativeStructure(1, 1, 0, Double.Na
N);
    DerivativeStructure y = p.value(xNaN);
    checkEquals("value DS", Double.NaN, y.getValue());
}
private static void Z317() {
    headline("Z317 ValueAtDS: coeffs have NaN → t_val is NaN");
    PolynomialFunction p = new PolynomialFunction(new double[] { Double.N
aN, 1.0, 2.0 });
    DerivativeStructure x = new DerivativeStructure(1, 1, 0, 0.3);
    DerivativeStructure y = p.value(x);
    checkEquals("value DS", Double.NaN, y.getValue());
}
private static void Z318() {
    headline("Z318 ValueAtDS: otherwise → finite Horner on DS");
    PolynomialFunction p = new PolynomialFunction(new double[] { 2.0, -
3.0, 4.0 }); // 2 - 3x + 4x^2
    DerivativeStructure x = new DerivativeStructure(1, 2, 0, 0.5);
    DerivativeStructure y = p.value(x);
    double expectedVal = 2.0 - 3.0 * 0.5 + 4.0 * 0.25; // 1.5
    double expectedD1 = -3.0 + 8.0 * 0.5; // 1.0

```

```

double expectedD2 = 8.0; // second derivative
checkEquals("value", expectedVal, y.getValue());
checkEquals("d1", expectedD1, y.getPartialDerivative(1));
checkEquals("d2", expectedD2, y.getPartialDerivative(2));
}
private static void Z319() {
    headline("Z319 Degree: |coeffs|≥1 → deg=|coeffs|-1");
    PolynomialFunction p = new PolynomialFunction(new double[] { 2.0, -
1.0, 3.0 });
    checkInt("degree", 2, p.degree());
}
private static void Z320() {
    headline("Z320 Derivative: |coeffs|=1 → d_coeffs=[0.0]");
    PolynomialFunction p = new PolynomialFunction(new double[] { 7.5 });
    double[] d = p.polynomialDerivative().getCoefficients();
    checkArrayEquals("d coeffs", new double[] { 0.0 }, d);
    // 补充: 一般导数 sanity
    PolynomialFunction p2 = new PolynomialFunction(new double[] { 2.0, -
3.0, 4.0 });
    double[] d2 = p2.polynomialDerivative().getCoefficients(); // {-3,8}
    checkArrayEquals("d coeffs general", new double[] { -3.0, 8.0 }, d2);
}
// ----- entry -----
public void Z3test() {
    System.out.println("=====PolynomialFunction • Z3 suite (Z301-
Z320) =====");
    Z301();
    Z302();
    Z303();
    Z304();
    Z305();
    Z306();
    Z307();
    Z308();
    Z309();
    Z310();
    Z311();
    Z312();
    Z313();
    Z314();
    Z315();
    Z316();
    Z317();
    Z318();

```

```

Z319();
Z320();
System.out.println("===== Done =====");
}
}
class PFtest {
    public static void main(String[] args) {
        // VT
        PFVT t1 = new PFVT();
        t1.Vtest();
        // FT
        PFFT t2 = new PFFT();
        // t2.Ftest();
        // Z3
        PFZ3 t3 = new PFZ3();
        // t3.Z3test();
    }
}

```

- Coverage report

1. Test case number: 20
2. V-Method testing branch coverage: 28.3%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
PolynomialFunction.java	30.7 %	59	133	192
> PFFT	0.0 %	0	10	10
> PFZ3	0.0 %	0	60	60
PolynomialFunction	28.3 %	17	43	60
evaluate(double[], double)	75.0 %	3	1	4
value(DerivativeStructure)	75.0 %	3	1	4
differentiate(double[])	83.3 %	5	1	6
PolynomialFunction(double[])	100.0 %	6	0	6
degree()		0	0	0
getCoefficients()		0	0	0
polynomialDerivative()		0	0	0
value(double)		0	0	0
> PFVT	70.0 %	42	18	60
> PFtest		0	0	0

3. V-Method testing instruction coverage: 30.3%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
PolynomialFunction.java	33.9 %	1,228	2,393	3,621
> PFZ3	0.3 %	3	982	985
> PFFT	0.3 %	3	905	908
PolynomialFunction	30.3 %	177	408	585
evaluate(double[], double)	86.1 %	31	5	36
value(DerivativeStructure)	89.4 %	42	5	47
differentiate(double[])	89.6 %	43	5	48
PolynomialFunction(double[])	100.0 %	38	0	38
degree()	100.0 %	6	0	6
getCoefficients()	100.0 %	5	0	5
polynomialDerivative()	100.0 %	7	0	7
value(double)	100.0 %	5	0	5
> PFtest	83.3 %	15	3	18
> PFVT	94.1 %	1,030	64	1,094

4. Fuzz testing branch coverage: 13.3%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
PolynomialFunction.java	9.4 %	18	174	192
PFVT	0.0 %	0	60	60
PFZ3	0.0 %	0	60	60
PolynomialFunction	13.3 %	8	52	60
evaluate(double[], double)	75.0 %	3	1	4
PolynomialFunction(double[])	83.3 %	5	1	6
degree()		0	0	0
value(double)		0	0	0
PFFT	100.0 %	10	0	10
PFTest		0	0	0

5. Fuzz testing instruction coverage: 12.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
PolynomialFunction.java	27.3 %	987	2,634	3,621
PFVT	0.3 %	3	1,091	1,094
PFZ3	0.3 %	3	982	985
PolynomialFunction	12.8 %	75	510	585
evaluate(double[], double)	86.1 %	31	5	36
PolynomialFunction(double[])	86.8 %	33	5	38
degree()	100.0 %	6	0	6
value(double)	100.0 %	5	0	5
PFTest	83.3 %	15	3	18
PFFT	98.1 %	891	17	908

6. Z3 branch coverage: 28.3%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
PolynomialFunction.java	26.6 %	51	141	192
PFFT	0.0 %	0	10	10
PFVT	0.0 %	0	60	60
PolynomialFunction	28.3 %	17	43	60
evaluate(double[], double)	75.0 %	3	1	4
value(DerivativeStructure)	75.0 %	3	1	4
differentiate(double[])	83.3 %	5	1	6
PolynomialFunction(double[])	100.0 %	6	0	6
degree()		0	0	0
getCoefficients()		0	0	0
polynomialDerivative()		0	0	0
value(double)		0	0	0
PFZ3	56.7 %	34	26	60
PFTest		0	0	0

7. Z3 instruction coverage: 30.3%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
PolynomialFunction.java	30.5 %	1,104	2,517	3,621
PFVT	0.3 %	3	1,091	1,094
PFFT	0.3 %	3	905	908
PolynomialFunction	30.3 %	177	408	585
evaluate(double[], double)	86.1 %	31	5	36
value(DerivativeStructure)	89.4 %	42	5	47
differentiate(double[])	89.6 %	43	5	48
PolynomialFunction(double[])	100.0 %	38	0	38
degree()	100.0 %	6	0	6
getCoefficients()	100.0 %	5	0	5
polynomialDerivative()	100.0 %	7	0	7
value(double)	100.0 %	5	0	5
PFTest	83.3 %	15	3	18
PFZ3	92.0 %	906	79	985